

Mi-8 Cheatsheet

Server Component Data Fetching

👉 Explanation:

Next.js App Router-এ default হচ্ছে Server Component। এখানে server থেকেই data fetch হয়, তাই client-side useEffect দরকার হয় না। এতে performance ও SEO দুটোই improve হয়।

```
async function getPosts() {  
  const res = await fetch('https://api.example.com/posts');  
  return res.json();  
}
```

```
export default async function PostsPage() {  
  const posts = await getPosts();  
  
  return posts.map(p => <p key={p.id}>{p.title}</p>);  
}
```

Default Caching (SSG)

👉 Explanation:

Next.js default ভাবে data cache করে রাখে। মানে build time-এ data fetch হয়ে যায় এবং পরে সেই cached data serve করে—fast loading।

```
fetch(url, { cache: 'force-cache' }) // default behavior
```

Dynamic Data Fetching (no-store)

👉 Explanation:

যখন সবসময় fresh data দরকার (যেমন dashboard), তখন cache বন্ধ করতে হয়।

```
fetch(url, { cache: 'no-store' })
```

ISR (revalidate)

👉 Explanation:

Static page হলেও নির্দিষ্ট সময় পর data update হয়—performance ও freshness এর balance।

```
fetch(url, { next: { revalidate: 10 } })
```

SSG vs ISR Decision

👉 Explanation:

Static data → SSG

সময় পর update → ISR

সবসময় live → no-store

👉 Real-world: blog = SSG, news = ISR, dashboard = dynamic

generateStaticParams

👉 Explanation:

Dynamic route গুলো আগে থেকেই generate করলে page load fast হয়।

```
export async function generateStaticParams() {  
  const posts = await fetch('...').then(res => res.json());  
  
  return posts.map(post => ({  
    id: post.id.toString(),  
  }));  
}
```

Suspense & Streaming

👉 Explanation:

পুরো page load হওয়ার আগে অংশ অংশ করে UI show করা—better user experience।

```
import { Suspense } from 'react';  
  
<Suspense fallback={<p>Loading...</p>}>  
  <Posts />  
</Suspense>
```

Server Actions

👉 Explanation:

API route ছাড়াই server-side logic লেখা যায়—clean architecture।

```
'use server';

export async function addPost(formData) {
  const title = formData.get('title');
}
```

Form Submission with Server Actions

👉 Explanation:

Form submit করলে সরাসরি server function call হয়—extra fetch দরকার নেই।

```
<form action={addPost}>
  <input name="title" />
  <button type="submit">Add</button>
</form>
```

useFormStatus (Pending State)

👉 Explanation:

Form submit হওয়ার সময় loading state সহজে handle করা যায়।

```
'use client';
import { useFormStatus } from 'react-dom';

function SubmitBtn() {
  const { pending } = useFormStatus();

  return <button disabled={pending}>
    {pending ? 'Adding...' : 'Add'}
  </button>
```

```
</button>;  
}
```

Revalidation (revalidatePath, revalidateTag)

👉 Explanation:

Data update হলে UI automatically refresh করতে সাহায্য করে।

```
import { revalidatePath } from 'next/cache';
```

```
revalidatePath('/posts');
```

Error Handling in Server Actions

👉 Explanation:

Production app-এ error properly handle না করলে app break করতে পারে।

```
try {  
  // logic  
} catch (err) {  
  throw new Error('Something went wrong');  
}
```

Lazy Loading (next/dynamic)

👉 Explanation:

Heavy component পরে load করে initial page speed বাড়ানো হয়।

```
import dynamic from 'next/dynamic';

const Heavy = dynamic(() => import('./HeavyComponent'), {
  loading: () => <p>Loading...</p>,
});
```

Authentication System Concept

👉 Explanation:

Authentication মানে user কে identify করা—login, logout, session maintain করা।

Register & Login (Credentials)

👉 Explanation:

User email/password দিয়ে account create ও login করতে পারে।

```
await signIn('credentials', {
  email,
  password,
});
```

Session Access (Server & Client)

👉 Explanation:

User login করা আছে কিনা জানার জন্য session use করা হয়।

```
const session = await getServerSession();
```

Protected Route

👉 Explanation:

Login ছাড়া access block করা—security এর জন্য গুরুত্বপূর্ণ।

```
if (!session) {  
  redirect('/login');  
}
```

OAuth Login (Google / GitHub)

👉 Explanation:

User social account দিয়ে login করতে পারে—UX improve হয়।

```
signIn('google');
```

Logout System

👉 Explanation:

User session destroy করে logout করা হয়।

```
signOut();
```

Data Fetch & Filtering

👉 Explanation:

Category বা condition অনুযায়ী data filter করা—common real-world logic।

```
const filtered = news.filter(n => n.category === selected);
```

Dynamic Route

👉 Explanation:

একই structure দিয়ে multiple dynamic page render করা যায়।

```
// app/news/[id]/page.jsx
```

Loading UI

👉 Explanation:

Data load হওয়ার সময় user কে feedback দেওয়া—better UX।

```
export default function Loading() {  
  return <p>Loading...</p>;  
}
```

Private Route Handling

👉 Explanation:

Authenticated user ছাড়া page access বন্ধ রাখা।

User Profile Update

👉 Explanation:

User নিজের data update করতে পারে—real-world feature।

```
await fetch('/api/user', {  
  method: 'PATCH',  
  body: JSON.stringify({ name }),  
});
```

Deployment (Vercel + ENV)

👉 Explanation:

Project live করার জন্য environment variable setup করা জরুরি।

```
NEXT_PUBLIC_API_URL=...  
AUTH_SECRET=...
```
